Value Function Approximation (VFA)

Agenda

- Introduction
- Value Function Approximation
- Incremental Methods

  e.g., Stochastic Gradient Decent

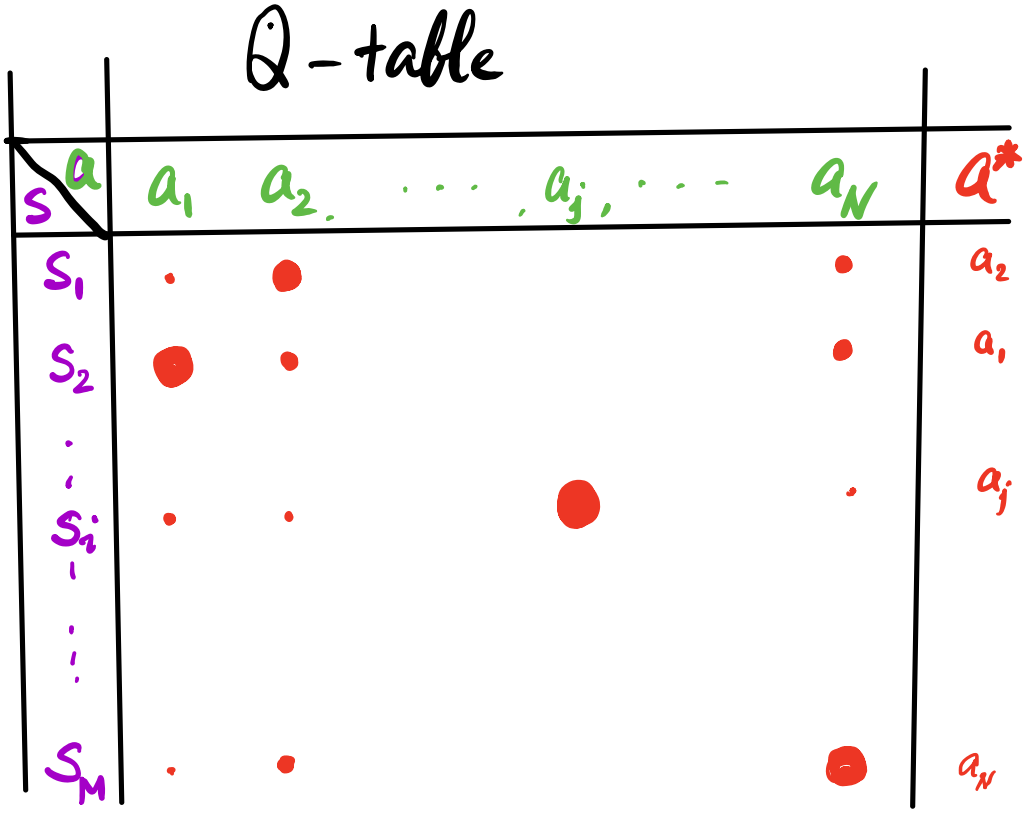
Introduction

- When # of States/Actions are moderate, lookup table is sufficient.

V-table

S	s_1, s_2, ...	s_M
$V_\pi(s)$	$V(s_1)$, $V(s_2)$,	$V(s_M)$

i.e. every state s has an entry $V(s)$.



i.e. every state-action pair (s, a)

has an entry $Q(s, a)$.

- In principle, one could find optimal policy by enemeration.

- Computational Cost:

$$|S|^{|A|}$$

(assuming $|A(s_1)| = \cdots = |A(s_M)| = |A|$)

$|A(s_1)|, \ldots, |A(s_M)|$ could be different.

- Examples

	# of States	size
Backgammon	10^{20}	✓
Go	10^{170}	too big
Helicopter	∞ (continuous).	too big

- Enormous State and/or Action space.

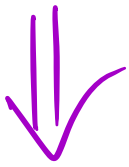
⇓

Tabular representation is insufficient !

- Too much storage in memory.
- Too slow to learn individually

- Solution ?

Value Function Approximation



Scale up Model-free Prediction and control.



How ?

Value Function Approximation

Value Function Approximation

- So far we have represented value function by a *lookup table*
  - Every state s has an entry $V(s)$
  - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
  - There are too many states and/or actions to store in memory
  - It is too slow to learn the value of each state individually
- Solution for large MDPs:
  - Estimate value function with *function approximation*

$$\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$$
$$\text{or } \hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$$

  - *Generalise* from seen states to unseen states
  - *Update* parameter $\mathbf{w}$ using MC or TD learning

- Estimate value function

$$\hat{v}(s, w) \approx v_\pi(s)$$

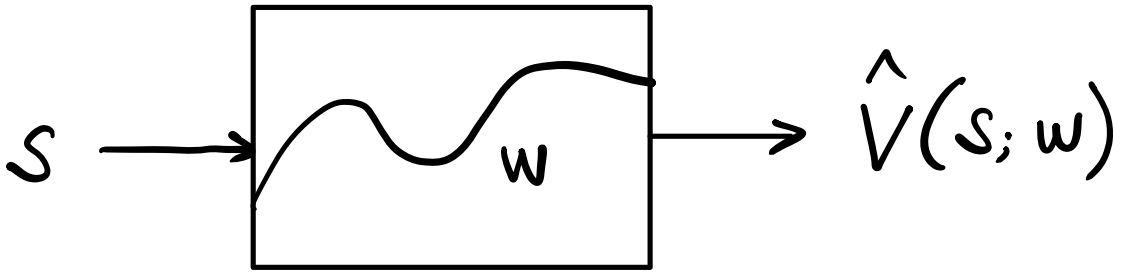
or

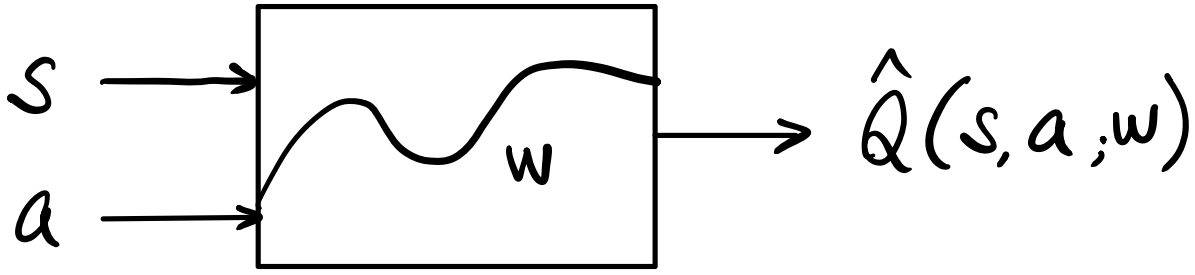
$$\hat{q}(s, a, w) \approx q_\pi(s, a)$$

- More compact representation
- Generalize from seen states/actions to unseen ones.
- Update parameter w using MC or TD learning

- Benefits of VFA:
  - Reduce memery in storage (only w)
  - Reduce computational cost
  - Reduce experience to learn

Types of VFA





$\Rightarrow$ Supervised Learning

Which Models ?

- Linear models

  i.e. linear combination of features

- Non-linear models
  - Decision trees
  - SVM
  - k nearest neighors (k NN)
  - Neaural Network
  - Basis expansion (spline/wavelet)
  - ... ...

Our focus here

- Differential w.r.t. w, e.g
  - Linear combination of features
  - Neaural Network
- Why ?

  Like to use GD, SGD, etc

  efficient, scalable

- Training method suitable for

  non-stationary, non-i.i.d.

Value Function Approximation

- Want: $\hat{V}(s; w) \approx v_\pi(s)$

  assumed to be known

- L^2-loss $= MSE$

$$J(w) \triangleq E_\pi\left(v_\pi(S) - \hat{V}(S; w)\right)^2$$

- Objective

$$\hat{w}^* = \arg\min_w J(w)$$

How to solve this ?

- Incremental Methods

  e.g. GD, SGD, Adam,

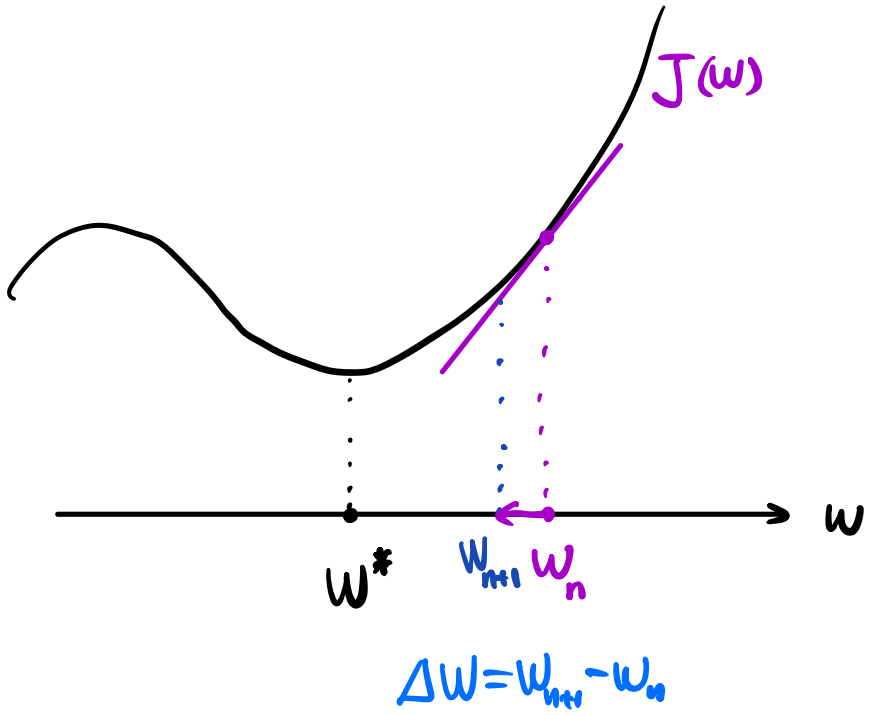
- Batch Methods.

Incremental Methods

- Gradient Decent (GD)
- Stochastic Gradient Decent.

Gradient Decent (GD)

- Demo:



$$w_{n+1} = w_n - \frac{\alpha}{2} \nabla_w J(w)$$

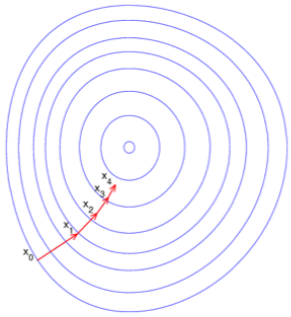
i.e.

$$\Delta w = -\frac{\alpha}{2} \nabla_w J(w)$$

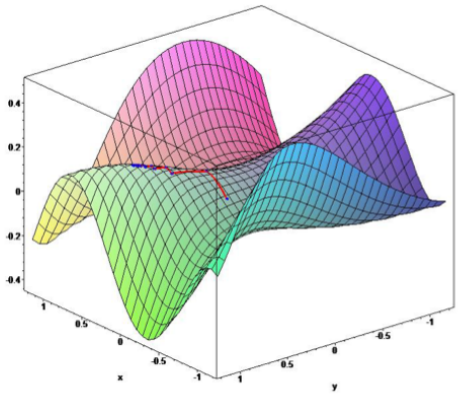
In general

- $J(w) \triangleq E_\pi\left(v_\pi(S) - \hat{V}(S; w)\right)^2$

- Gradient of $J(w)$:



$$\nabla_w J(w) = \begin{pmatrix} \frac{\partial J(w)}{\partial w_1} \\ \vdots \\ \frac{\partial J(w)}{\partial w_n} \end{pmatrix}$$



$$= -2 E_\pi\left[\left(v_\pi(S) - \hat{V}(S; w)\right) \nabla_w \hat{V}(S; w)\right]$$

- Adjust w in direction of -ve gradient.

$$\Delta w = -\frac{1}{2}\alpha \nabla_w J(w)$$

$$= \alpha \mathbb{E}_\pi \left[\left(v_\pi(S) - \hat{V}(S; w) \right) \nabla_w \hat{V}(S; w) \right]$$

step size

- $\nabla_w J(w)$ can be computationally expensive (involving $\mathbb{E}_\pi$).

Stochastic Gradient Decent (SGD)

- SGD samples the gradient:

$$\Delta w = -\frac{1}{2}\alpha \nabla_w J(w)$$

$$= \alpha \left(v_\pi(S) - \hat{V}(S; w)\right) \nabla_w \hat{V}(S; w)$$

- $E_\pi(SGD) = GD$

Example: Linear VFA.

- Represent state by a "feature vector":

$$x(s) = \begin{pmatrix} x_1(s) \\ \vdots \\ x_n(s) \end{pmatrix}$$

- Represent value by a linear combination of features:

$$V(s; w) = x(s)^T w$$

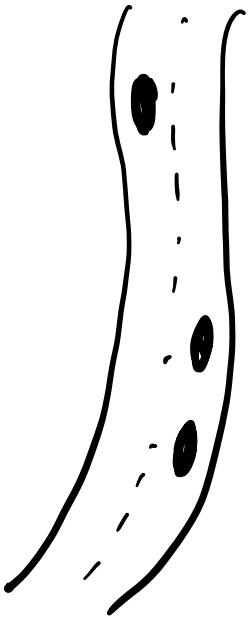
- Examples of feature vectors.

  - Blackjack

$$x(s) = \begin{pmatrix} \text{Player sum} \\ \text{Dealer card value} \\ \mathbb{I}\{\text{Usable Ace}\} \end{pmatrix}$$

  - Self driving

$$x(s) = \begin{pmatrix} \text{dist to central line} \\ \text{dist to fence} \\ \text{dist to car in front} \\ \vdots \end{pmatrix}$$



- Objective function

$$J(w) = E_\pi \left(v_\pi(s) - x(s)^T w \right)^2$$

$$\nabla_w J(w) = -2 E_\pi \left(v_\pi(s) - x(s)^T w \right) x(s)$$

- SGD

$$\Delta w = \alpha \left(v_\pi(s) - x(s)^T w \right) x(s)$$

update = (step size) x (Prediction error) x (feature value)

- $J(w)$ is quadratic in w (Convex)

  $\Rightarrow$ SGD converges to global optimum.

Table lookup $\subseteq$ Linear VFA.

- Lookup Table

States	$s_1, s_2, \ldots, s_n$
Values	$v(s_1), v(s_2), \ldots, v(s_n)$ $\|\|$ $w_1 \quad w_2 \quad \quad w_n$

- $$\chi(s) = \begin{pmatrix} \mathbb{I}\{s = s_1\} \\ \vdots \\ \mathbb{I}\{s = s_n\} \end{pmatrix}$$

- $$\hat{v}(s; w) = \begin{pmatrix} \mathbb{I}\{s = s_1\} \\ \vdots \\ \mathbb{I}\{s = s_n\} \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ \vdots \\ w_n \end{pmatrix} = \chi(s)^T w$$

Prediction with VFA

(policy evaluation)

An oracle $v_\pi(S)$ is unknown

We substitute a target for $v_\pi(S)$.

- MC target G_t:

$$\Delta w = \alpha \left[G_t - \hat{V}(S_t; w) \right] \nabla_w \hat{V}(S_t; w)$$

- TD(0) target $R_{t+1} + \gamma \hat{V}(S_{t+1}, w)$:

$$\Delta w = \alpha \left[R_{t+1} + \gamma \hat{V}(S_{t+1}, w) - \hat{V}(S_t; w) \right] \nabla_w \hat{V}(S_t; w)$$

- TD(λ) target G_t^λ

$$\Delta w = \alpha \left[G_t^\lambda - \hat{V}(S_t; w) \right] \nabla_w \hat{V}(S_t; w)$$

#

MC with VFA

- Update $v_\pi(S_t)$ after each episode

$$S_1, A_1, R_2, S_2, A_2, R_3, \ldots, S_{T-1}, A_{T-1}, R_T$$

- Calculate

$$(S_1, G_1), (S_2, G_2), \ldots, (S_{T-1}, G_{T-1})$$

- MC policy evaluation

$$\Delta w = \alpha \left[G_t - \hat{V}(S_t; w) \right] \nabla_w \hat{V}(S_t; w)$$

$$\overset{\text{linear}}{=} \alpha \left[G_t - \hat{V}(S_t; w) \right] x(S_t)$$

TD(0) with VFA

- Update $v_\pi(S_t)$ after each time-step

$$S_t, A_t, R_{t+1}, S_{t+1}$$

- $v_\pi(S_t)$ is approximated by TD(0)-target:

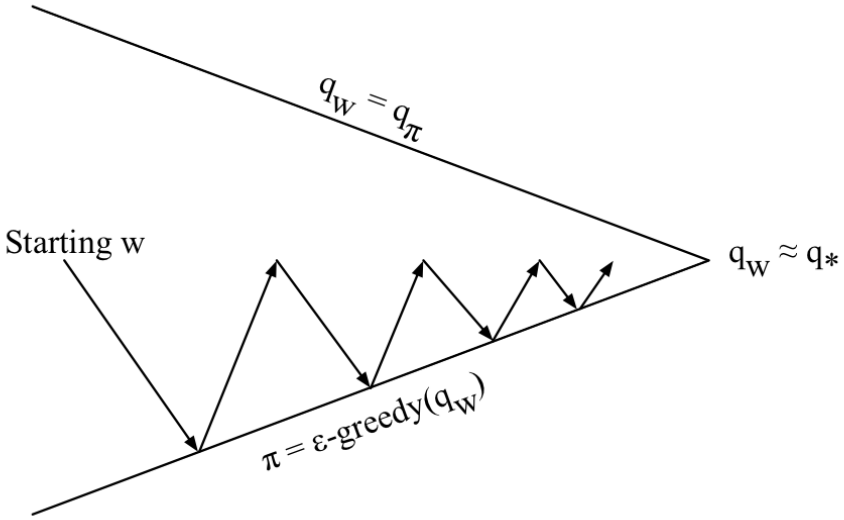
$$R_{t+1} + \gamma \hat{V}(S_{t+1}; w)$$

- TD(0) policy evaluation

$$\Delta w = \alpha \left[R_{t+1} + \gamma \hat{V}(S_{t+1}; w) - \hat{V}(S_t; w) \right] \nabla_w \hat{V}(S_t; w)$$

$$\overset{\text{linear}}{=} \alpha \left[R_{t+1} + \gamma \hat{V}(S_{t+1}; w) - \hat{V}(S_t; w) \right] x(S_t)$$

Control with VFA



- Policy evaluation

  (Action-Value Function Approx, AVFA)

$$\hat{q}(s, a; w) \approx q_\pi(s, a)$$

- Policy improvement

  ε-greedy policy improvement.

Policy evaluation (AVFA)

- Want: $\hat{q}(S, a; w) \approx q_\pi(S, a)$

  assumed to be known

- L^2-loss $=$ MSE

$$J(w) \triangleq E_\pi\left(q_\pi(S, a) - \hat{q}(S, a; w)\right)^2$$

- Objective

$$\hat{w}^* = \arg\min_w J(w)$$

- GD:

$$\Delta w = -\frac{1}{2}\alpha \nabla_w J(w)$$

$$= \alpha \mathbb{E}_\pi \left[\left(q_\pi(s,a) - \hat{q}(s,a;w) \right) \nabla_w \hat{q}(s,a;w) \right]$$

- SGD:

$$\Delta w = \alpha \left(q_\pi(s,a) - \hat{q}(s,a;w) \right) \nabla_w \hat{q}(s,a;w)$$

#

Example: Linear AVFA.

- Represent state by a "feature vector":

$$x(S,a) = \begin{pmatrix} x_1(S,a) \\ \vdots \\ x_n(S,a) \end{pmatrix}$$

- Represent action-value by a linear combination of features:

$$\hat{q}(S,a,w) = x(S,a)^T w$$

- SGD:

$$\Delta w = \alpha \left(q_\pi(S,a) - \hat{q}(S,a,w) \right) x(S,a)$$

Policy Improvement

An oracle $q_\pi(S, a)$ is unknown.

We substitute a target for $q_\pi(S, a)$.

- MC target G_t:

$$\Delta w = \alpha \left[G_t - \hat{q}(S_t, A_t; w) \right] \nabla_w \hat{q}(S_t, A_t; w)$$

- SARSA(0) target $R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; w)$:

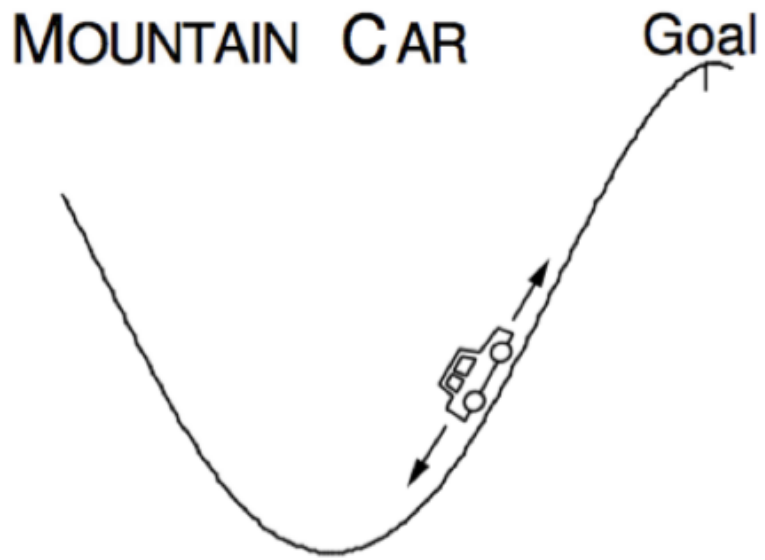
$$\Delta w = \alpha \left[R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; w) - \hat{q}(S_t, A_t; w) \right] \nabla_w \hat{q}(S_t, A_t; w)$$

Mini Summary

- SGD:
  - simple & appealing.
  - not sample efficient.
  - Need more efficient algorithms.
- Linear combinations of features.
  - Effective for simple problems.
  - Non-linear models needed e.g. DNN.

An example:

Mountain Car



Example 10.1: Mountain Car Task Consider the task of driving an underpowered car up a steep mountain road, as suggested by the diagram in the upper left of Figure 10.1. The difficulty is that gravity is stronger than the car's engine, and even at full throttle the car cannot accelerate up the steep slope. The only solution is to first move away from the goal and up the opposite slope on the left. Then, by applying full throttle the car can build up enough inertia to carry it up the steep slope even though it is slowing down the whole way. This is a simple example of a continuous control task where things have to get worse in a sense (farther from the goal) before they can get better. Many control methodologies have great difficulties with tasks of this kind unless explicitly aided by a human designer.

The reward in this problem is -1 on all time steps until the car moves past its goal position at the top of the mountain, which ends the episode. There are three possible actions: full throttle forward ($+1$), full throttle reverse (-1), and zero throttle (0). The car moves according to a simplified physics. Its position, x_t, and velocity, $\dot{x}_t$, are updated by

$$x_{t+1} \doteq \mathit{bound}\big[x_t + \dot{x}_{t+1}\big]$$

$$\dot{x}_{t+1} \doteq \mathit{bound}\big[\dot{x}_t + 0.001 A_t - 0.0025\cos(3x_t)\big],$$

where the *bound* operation enforces $-1.2 \leq x_{t+1} \leq 0.5$ and $-0.07 \leq \dot{x}_{t+1} \leq 0.07$. In addition, when x_{t+1} reached the left bound, $\dot{x}_{t+1}$ was reset to zero. When it reached the right bound, the goal was reached and the episode was terminated. Each episode started from a random position $x_t \in [-0.6, -0.4)$ and zero velocity. To convert the two continuous state variables to binary features, we used grid-tilings as in Figure 9.9. We used 8 tilings, with each tile covering 1/8th of the bounded distance in each dimension, and asymmetrical offsets as described in Section 9.5.4.[1] The feature vectors $\mathbf{x}(s,a)$ created by tile coding were then combined linearly with the parameter vector to approximate the action-value function:

$$\hat{q}(s,a,\mathbf{w}) \doteq \mathbf{w}^\top \mathbf{x}(s,a) = \sum_{i=1}^{d} w_i \cdot x_i(s,a), \tag{10.3}$$

for each pair of state, s, and action, a.

- State

$$S_t = (x_t, \dot{x}_t) = (\text{position}, \text{velocity})$$

- Action

$$A_t = \begin{cases} 1 & \text{full throttle forward} \\ 0 & \text{throttle} \\ -1 & \text{full throttle reverse} \end{cases}$$

- Reward

$$R_t = \begin{cases} 0 & \text{goal} \\ -1 & \text{otherwise} \end{cases}$$

- Dynamics :
  - Velocity :

$$x_{t+1} \doteq x_t + \dot{x}_{t+1} \in [-1.2, 0.5]$$

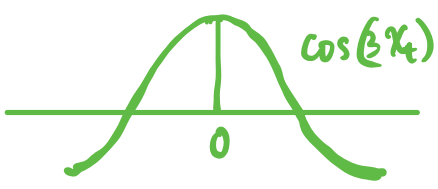
$$\Leftrightarrow \quad \dot{x}_{t+1} = \frac{x_{t+1} - x_t}{(t+1) - t}$$

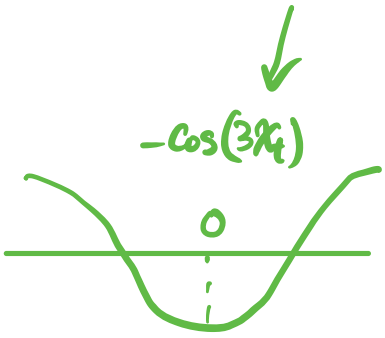
  - Acceleration :

$$\dot{x}_{t+1} \doteq \dot{x}_t + 0.001\, A_t - 0.0025 \cos(3x_t)$$

$$\in [-0.07, 0.07]$$

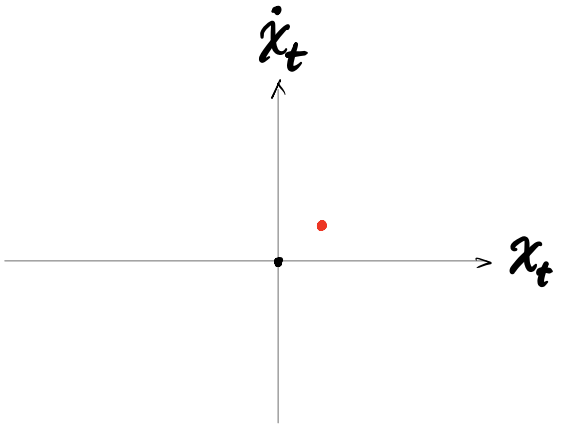
$$\Leftrightarrow \quad \ddot{x}_t \doteq \frac{\dot{x}_{t+1} - \dot{x}_t}{(t+1) - t} = 0.001\, A_t - 0.0025 \cos(3x_t)$$

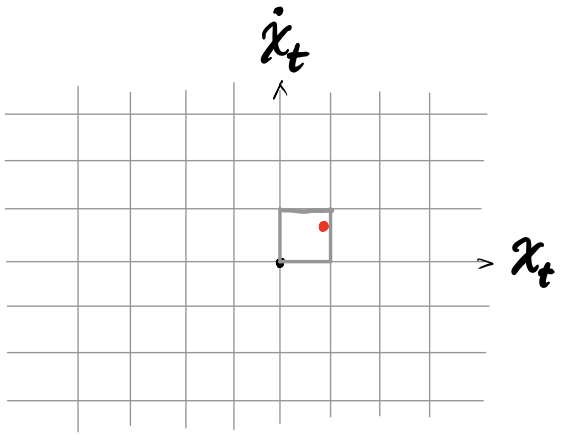


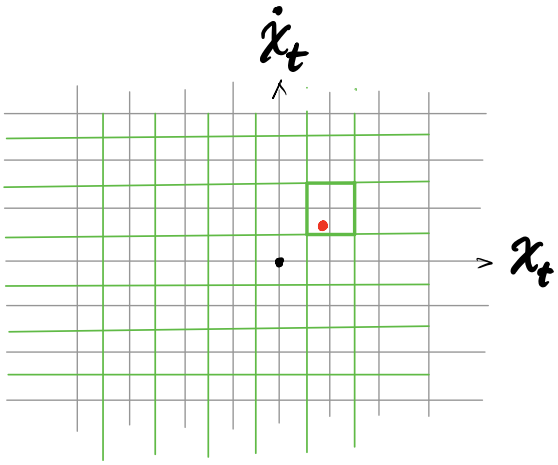


- Feature vectors.
  - Coarse coding (e.g tile-coding)

(Very similiar to kNN, identifying hot spots.)







$$\chi(s,a) = \begin{pmatrix} \chi_1(s,a) \\ \vdots \\ \chi_n(s,a) \end{pmatrix}$$

e.g. $n = 4 \times 4 \times 4 \times 3 = 64 \times 3 = 192$

or

$n = 10 \times 10 \times 10 \times 3 = 3000$

- Linear AVFA:

$$\hat{q}(s,a,w) \doteq w^\top \chi(s,a)$$

$$= \sum_{i=1}^{d} w_i \chi_i(s,a)$$

9.5.4 Tile Coding

Tile coding is a form of coarse coding for multi-dimensional continuous spaces that is flexible and computationally efficient. It may be the most practical feature representation for modern sequential digital computers.

In tile coding the receptive fields of the features are grouped into partitions of the state space. Each such partition is called a *tiling*, and each element of the partition is called a *tile*. For example, the simplest tiling of a two-dimensional state space is a uniform grid such as that shown on the left side of Figure 9.9. The tiles or receptive field here are squares rather than the circles in Figure 9.6. If just this single tiling were used, then the state indicated by the white spot would be represented by the single feature whose tile it falls within; generalization would be complete to all states within the same tile and nonexistent to states outside it. With just one tiling, we would not have coarse coding but just a case of state aggregation.

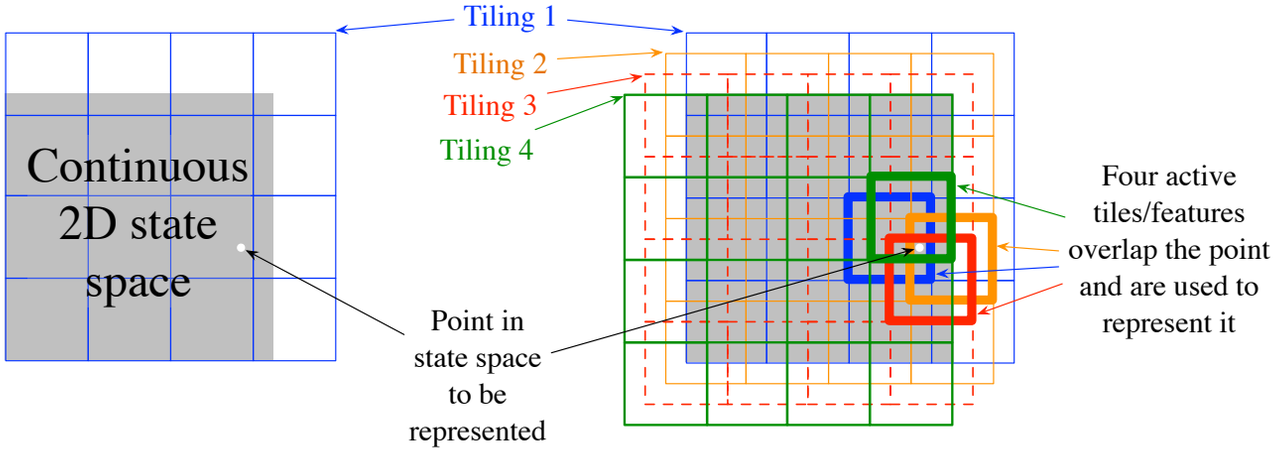


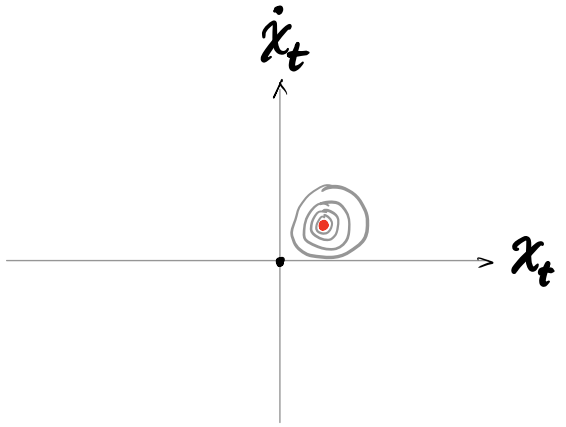
Figure 9.9: Multiple, overlapping grid-tilings on a limited two-dimensional space. These tilings are offset from one another by a uniform amount in each dimension.

To get the strengths of coarse coding requires overlapping receptive fields, and by definition the tiles of a partition do not overlap. To get true coarse coding with tile coding, multiple tilings are used, each offset by a fraction of a tile width. A simple case with four tilings is shown on the right side of Figure 9.9. Every state, such as that indicated by the white spot, falls in exactly one tile in each of the four tilings. These four tiles correspond to four features that become active when the state occurs. Specifically, the feature vector $\mathbf{x}(s)$ has one component for each tile in each tiling. In this example there are $4 \times 4 \times 4 = 64$ components, all of which will be 0 except for the four corresponding to the tiles that s falls within. Figure 9.10 shows the advantage of multiple offset tilings (coarse coding) over a single tiling on the 1000-state random walk example.

An immediate practical advantage of tile coding is that, because it works with partitions, the overall number of features that are active at one time is the same for any state. Exactly one feature is present in each tiling, so the total number of features present is always the same as the number of tilings. This allows the step-size parameter, α, to be set in an easy, intuitive way. For example, choosing $\alpha = \frac{1}{n}$, where n is the number

- Kernel method:

  e.g. radial basis functions



$$\chi_i(s) \doteq \exp\left(-\frac{\|s - c_i\|^2}{2\sigma_i^2}\right)$$

Results

Linear SARSA with coarse coding

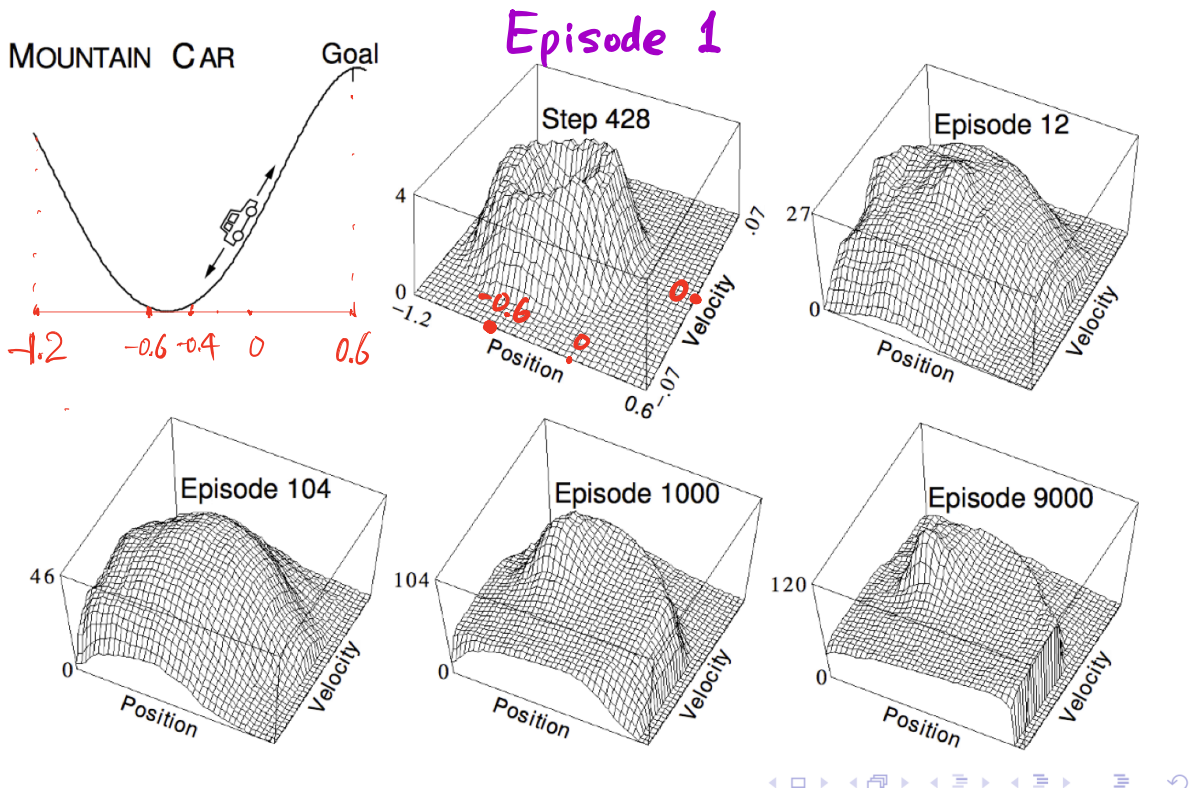


Figure 10.1 shows what typically happens while learning to solve this task with this form of function approximation.[2] Shown is the negative of the value function (the *cost-to-go* function) learned on a single run. The initial action values were all zero, which was optimistic (all true values are negative in this task), causing extensive exploration to occur even though the exploration parameter, ε, was 0. This can be seen in the middle-top panel of the figure, labeled "Step 428". At this time not even one episode had been completed, but the car has oscillated back and forth in the valley, following circular trajectories in state space. All the states visited frequently are valued worse than unexplored states, because the actual rewards have been worse than what was (unrealistically) expected. This continually drives the agent away from wherever it has been, to explore new states, until a solution is found.

Linear SARSA with radial basis functions

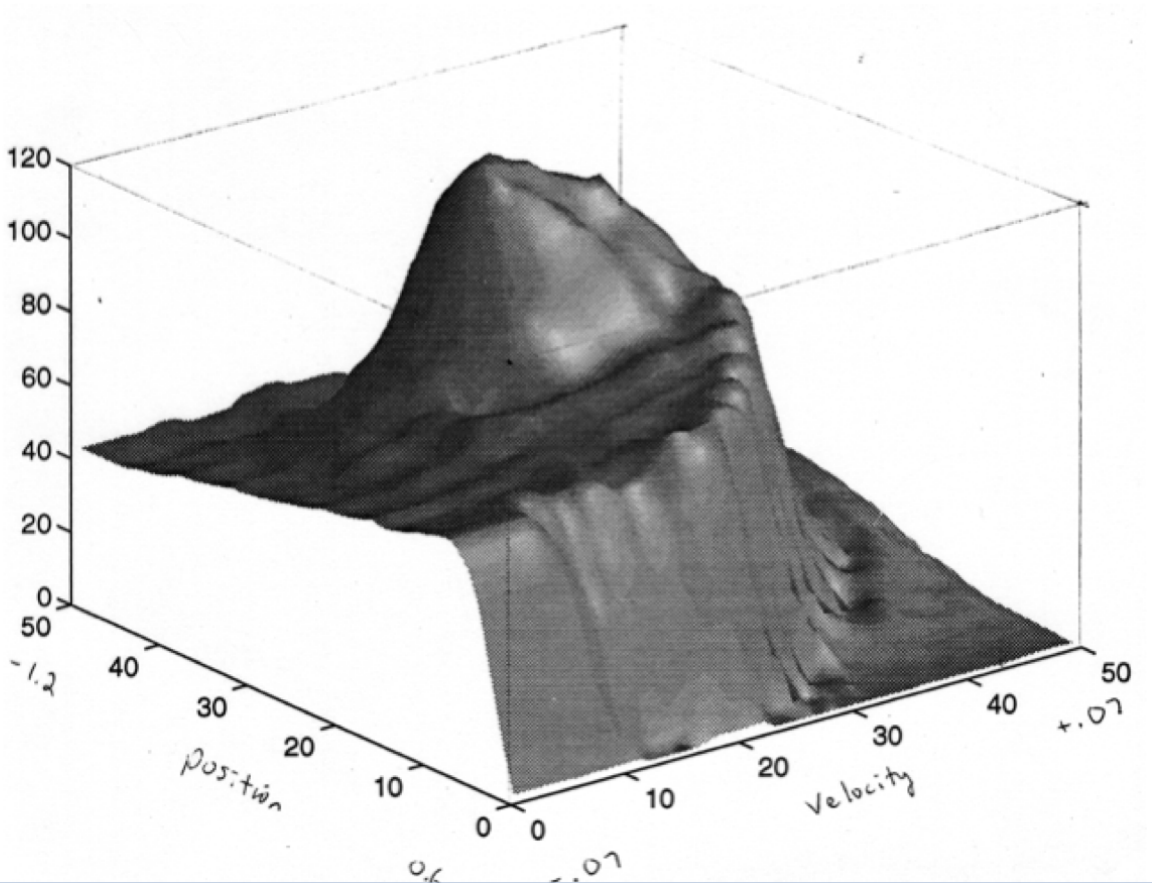


Figure 10.2 shows several learning curves for semi-gradient Sarsa on this problem, with various step sizes.

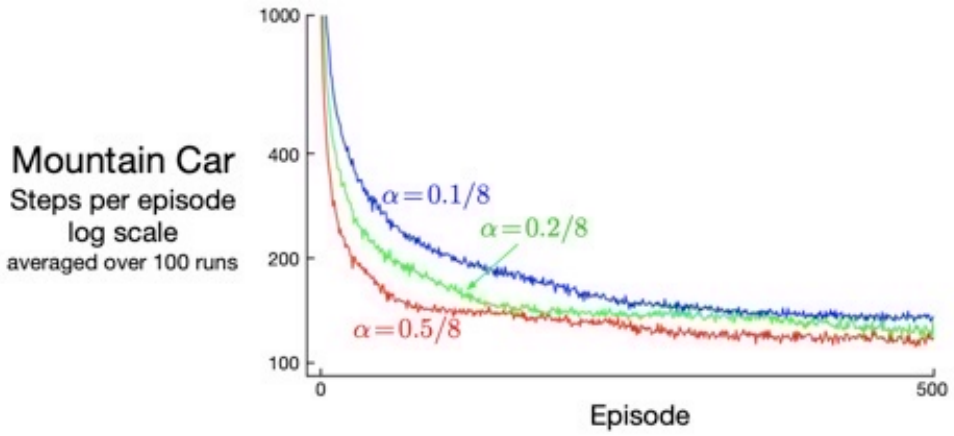


Figure 10.2: Mountain Car learning curves for the semi-gradient Sarsa method with tile-coding function approximation and ε-greedy action selection. ■